Binary to C++ With Security Vulnerabilities Activity Guidelines and Rubric

CS 410 Binary to C++ With Security Vulnerabilities Activity Guidelines and Rubric

Overview

In previous activities, you converted files using languages including binary, assembly, and C++. You have successfully completed a full conversion of legacy code, from binary to C++. In this activity, you will convert legacy binary code to C++, examine the code for security vulnerabilities, fix the identified security vulnerabilities, and report on those vulnerabilities. This is a key step toward the successful completion of Project Two, which is centered on fixing security vulnerabilities. The coding for this assignment will be performed in Codio. You will submit the completed Binary to C++ With Security Vulnerabilities Activity Template Word Document and a new binary file and CPP file.

Prompt

Specifically, you must address the following rubric criteria:

1. Convert the binary file to assembly code.
    - These files can be found in the Software Reverse Engineering Playground in the Module Six file folder in Codio.
2. Explain the functionality of the blocks of assembly code.
    - Use the Binary to C++ With Security Vulnerabilities Activity Template to complete this step.
3. Convert the assembly code to binary.
    - Create a new binary file for submission.
4. Convert the assembly code to C++ code.
    - Use the Binary to C++ With Security Vulnerabilities Activity Template to complete this step.
    - Compile the C++ code in the Eclipse integrated development environment.
5. Identify the security vulnerabilities within the C++ code.
    - Comment within the C++ code to indicate where the security vulnerabilities are identified.
6. Fix the identified security vulnerabilities within the C++ code.
    - Correct the C++ code to fix the security vulnerabilities.
7. Explain how the updated C++ code fixes the identified vulnerabilities.
    - Comment within the C++ code to indicate how the security vulnerabilities were fixed.

What to Submit

Binary to C++ With Security Vulnerabilities Activity Template

This should be a Word document. Use the Binary to C++ With Security Vulnerabilities Activity Template to convert the legacy binary file into assembly, explain the functionality of the assembly code, and convert the assembly code to C++ code.

C++ File

This file includes your comments on the identified security vulnerabilities, the fixes, and the comments about how the security vulnerabilities were fixed. This file is needed to ensure that the code, identified

vulnerabilities, fixes, and comments are correct.

Binary File

This file is needed to run the application.

AI Usage

If you use gen AI tools to support your work on this assignment, be sure to follow these AI usage guidelines. You must acknowledge your use of these tools in your work. Guidelines on how to cite AI tools can be found in this Shapiro Library guide.

Binary to C++ With Security Vulnerabilities Activity Rubric

Criteria	Exceeds Expectations	Meets Expectations	Partially Meets Expectations	Does Not Meet Expectations	Value
Binary to Assembly Conversion	N/A	Converts binary file into assembly code (100%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include converting the binary file into assembly code (85%)	Does not attempt criterion (0%)	9
Assembly Functionality	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Explains the functionality of the blocks of assembly code with minimal errors and adequate detail (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include explaining the functionality of the blocks of assembly code with minimal errors and richer detail (55%)	Does not attempt criterion (0%)	13
Assembly to Binary Conversion	N/A	Converts the assembly code to binary (100%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include converting the assembly code to binary (85%)	Does not attempt criterion (0%)	9
Assembly to C++ Conversion	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Converts each block of assembly code to C++ code with minimal errors (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include converting each block of assembly code to C++ code with fewer errors (55%)	Does not attempt criterion (0%)	10
Identify Security Vulnerabilities	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Identifies all of the security vulnerabilities within the C++ code with minimal errors (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include identifying all of the security vulnerabilities within the C++ code with fewer errors (55%)	Does not attempt criterion (0%)	18
Fix Security Vulnerabilities	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner	Fixes the identified security vulnerabilities within the C++ code with minimal errors (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement	Does not attempt criterion (0%)	18

	sophisticated, or creative manner (100%)	code with minimal errors (85%)	omissions; areas for improvement may include fixing the identified security vulnerabilities within the C++ code with fewer errors (55%)		
Security Vulnerabilities Explanation	Exceeds expectations in an exceptionally clear, insightful, sophisticated, or creative manner (100%)	Explains how the updated C++ code fixes the identified security vulnerabilities with minimal errors (85%)	Shows progress toward meeting expectations, but with errors or omissions; areas for improvement may include explaining how the updated C++ code fixes the identified security vulnerabilities with fewer errors (55%)	Does not attempt criterion (0%)	18
Clear Communication	Exceeds expectations with an intentional use of language that promotes a thorough understanding (100%)	Consistently and effectively communicates in an organized way to a specific audience (85%)	Shows progress toward meeting expectations, but communication is inconsistent or ineffective in a way that negatively impacts understanding (55%)	Shows no evidence of consistent, effective, or organized communication (0%)	5
				Total:	100%

Reflect in ePortfolio Download Print Open with docReader

Activity Details

- Task: View this topic